

**TECHNISCHE
UNIVERSITÄT
DRESDEN**

Faculty of Electrical and Computer Engineering

Institute of Circuits and Systems

Chair of Highly-Parallel VLSI Systems and Neuro-Microelectronics

PROJECT WORK

Analysis and Implementation of an Action Potential Neural Signal Decorrelation Engine Using a Linear Neural Network

Student: Wei Mo

Enrollment number: 5198168

Born on: 26.07.2000, Huzhou

Supervisor: Dipl.-Ing. Liyuan Guo

1st Examiner: Prof. Dr.-Ing. habil. Christian Mayr

2nd Examiner:

Submit on: July 15, 2025

Abstract

The action potential signal plays a crucial role in the realm of neuro-prosthetic. Practically, modern neural signal recording usually requires long-term, high-channel-count data acquisition inside the brain, which is limited by transmission bandwidth and power consumption. To address these challenges, efficient compression techniques are essential to reduce data amount while preserving signal features. Decorrelation, as the first and key stage of neural signal compression, is explored in this thesis.

A lightweight linear neural network (LNN)-based signal decorrelation method is developed and implemented in both software and hardware. The model is trained and optimized, tuning the hyper-parameters such as taps and training epochs, to improve decorrelation accuracy for different signal types and noise conditions. To suit for hardware implementation, the parameters in the model are quantized to fixed-point doubles, balancing memory consumption and prediction accuracy. Multiple LNN modules are designed to work in parallel for multi-channel data, and the system supports configurable number of channels, making it adaptable to various application requirements. The functionality of hardware design is verified by simulation. Synthesis results under a 28nm technology node shows that the proposed design achieves efficient decorrelation capability with moderate area and power overhead, providing a practical solution for real-time low-power neural signal compression in next-generation brain implants.

The main work is as follows:

- Study on current algorithms for neural signal decorrelation.
- Design and optimization of a software-based LNN-based neural signal decorrelation algorithm, and according analysis for hardware implantation.
- Synthesizable hardware implementation of the LNN decorrelator in RTL.
- Module functionality verification, area and power consumption analysis.

Statement of Authorship

I hereby declare that I completed this thesis with the topic

**Analysis and Implementation of an Action Potential Neural Signal
Decorrelation Engine Using a Linear Neural Network**

on my own and that information which has been directly or indirectly taken from other sources has been noted as such. Neither this, nor a similar work, has been published or presented to an examination committee.

Dresden, July 15, 2025

Contents

List of Figures	v
List of Tables	vii
List of Abbreviations	ix
1 Introduction	1
1.1 Background	1
1.2 Basic Concepts	2
1.2.1 Decorrelation	2
1.2.2 Entropy Encoding	4
1.3 Datasets	5
1.4 Thesis Structure	6
2 Overview of Compression Algorithm	7
2.1 LNN-Based Decorrelation	7
2.1.1 LNN Model	8
2.1.2 Training Process	10
2.1.3 Inference Process	11
2.1.4 Decoding	12
2.2 Entropy Encoding	13
3 Software Implementation	15
3.1 Algorithm Realization	17
3.1.1 LNN-Based Decorrelation Implementation	17
3.1.2 Golomb Encoding Implementation	18
3.2 Performance Optimization	19
3.2.1 Optimal Epochs	20
3.2.2 Optimal Taps	22
3.3 Memory Recourse Optimization	23
3.4 Real-Time Processing	25

4	Hardware Implementation	27
4.1	Hardware Design	27
4.1.1	Multi-Channel Processing	28
4.1.2	Decorrelation Unit	29
4.2	Functionality Verification	31
4.3	Performance Analysis	31
5	Conclusion and Related Works	33
5.1	Summary	33
5.2	Discussion	34
5.3	Related Works	34
A	Golomb Coding	37
A.1	Encoding	37
A.2	Decoding	38
	Bibliography	43

List of Figures

1.1	LLC Workflow	2
1.2	Spiking and Non-Spiking Areas	5
2.1	Neural Signal Compression	7
2.2	LNN-Based Decorrelation	8
2.3	Neuron	9
2.4	FIR Filter	10
3.1	Software Design	15
3.2	Algorithm Optimization	16
3.3	Loss (MSE)	17
3.4	Statistical Distribution after LNN-Based Decorrelation	18
3.5	Epoch Optimization for Different Signals	21
3.6	Tap Optimization for Different Signals	22
3.7	BW Optimization for Different Signals	24
4.1	Hardware Implementation	27
4.2	Top Module of the LNN-Based Decorrelator	28
4.3	Hybrid Decorrelation Unit	30
4.4	Verification Scheme	31
4.5	Area and Power Consumption	32
A.1	Golomb Encoding	37

List of Tables

3.1	Choice of Adaptive Parameter m for Golomb Encoding	19
3.2	Construction of Truncated Parameters	23
3.3	Performance of Optimized Algorithm	26

List of Abbreviations

BMI	Brain-Machine Interface
AP	Action Potential
LFP	Local Field Potential
DSP	Data Signal Processing
LLC	Lossless Compression
NLLC	Nearly Lossless Compression
SSR	Space Saving Ratio
DPCM	Differential Pulse-Code Modulation
LNN	Linear Neural Network
MSE	Mean Square Error
NL	Noise Level
NN	Neural Network
FIR	Finite Impulse Response
LR	Learning Rate
LF	Loss Function
GD	Gradient Descent
LMS	Least Mean-Square
ADC	Analog-Digital Converter
GC	Golomb Coding
AC	Arithmetic Coding
BW	Bit-Width
SRAM	Static Random-Access Memory
ECoG	Electrocorticography

1 Introduction

1.1 Background

Brain-machine interface (BMI), as a key technology to realize the connection between human brain and external devices, has drawn wide attention in recent years. BMI realizes direct manipulation of external devices by the brain. The development of this technology provides possibilities for the rehabilitation of paralyzed patients, the monitoring of brain diseases and the realization of intelligent interactive systems. [1] The neural signal can be acquired using intra- or extra-cellular electrodes. [2] The captured neural signal includes action potential (AP), local field potential (LFP) and others components such as background noise, which records the information of electric fields generated by the activity of neurons in the brain by the electrodes on the scalp. [3]

Due to the development of BMI systems, neural signal acquisition techniques with multiple channels are progressively popular nowadays. Practically, the real-time processing and transmission of multi-channel neural signals demand significantly large amount of system resource, including system bandwidth, memory capacity and power consumption. In this case, the traditional digital signal processing (DSP) algorithms and devices can hardly satisfy the requirements of efficient real-time processing.

By removing redundant information, data compression, as an effective method to solve this problem, has the ability to reduce the cost of data storage and transmission. [4] For neural signals, especially for the AP components, lossless compression (LLC) are preferred to lossy compression, which means that the signals can be completely reconstructed after decompression, to preserve the information carried by the neural signals. In the mean time, because of the relatively lower complexity, nearly-lossless compression (NLLC) can be applied to the spikes of AP, sacrificing some of the information to achieve higher space saving ratio (SSR). [5] LLC and NLLC ensure both effective data reduction and high fidelity of information, which adapts the compressed signals for real-time DSP. Thus, the development and optimization of efficient compression algorithms for neural signals, as well as its low-power integrated circuit implementation, are the core issues to fulfill the requirements of BMI systems.

1.2 Basic Concepts

The efficient processing of signal is the key to LLC, which minimizes the amount of data while maintaining the fidelity.

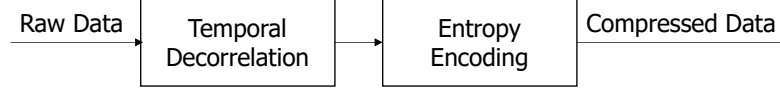


Figure 1.1: LLC Workflow

LLC usually consists of two main steps: decorrelation and encoding. First, the data in each channel is temporally decorrelated to minimize the entropy. And then the entropy encoding process compresses the decorrelated data according to the probability distribution of data values, as shown in Figure 1.1. The basic concepts mentioned in the thesis are explained in the following sections.

1.2.1 Decorrelation

For single-channel acquisition of raw neural signal, the values of adjacent samples can be relatively close to each other, which means the variation of voltage can be slow across a time window, namely self-correlation. It leads to temporal redundancies of the captured data. For multiple channels, since the electrical signal collected by each neural probe is the combination of the potentials from all of the nearby neurons, so there are also spacial redundancies between the data in different channels.

The process of decorrelation is to eliminate the temporal or spatial correlation of the signals to reduce data redundancy. Specially, the temporal decorrelation algorithms, such as differential pulse-code modulation (DPCM) and temporal decorrelation based on linear neural network (LNN), calculate sequentially the difference between the acquired value and the one predicted with the former samples. [6]

After decorrelation, the distribution of data will be centralized around 0.

1) 1st-Order DPCM

1st-order DPCM is a commonly used decorrelation method, which calculates the difference between each 2 adjacent samples, to catch the temporal variation of signal so that the temporal redundancy can be reduced. 1st-order DPCM is a straightforward technique for decorrelation, especially effective for smooth signals.

For input series x with index n , the difference is calculated as

$$d[n] = x[n] - x[n - 1] \quad (1.1)$$

in which $x[n]$ is the value of the current data sample, and $x[n-1]$ is the previous one.

Since the decorrelated data $d[n]$ are usually within a smaller range than $x[n]$ and centered more closely around 0, so that the mean-square error (MSE) of the signal can be reduced. Hence, fewer bits are required for encoding, which benefits entropy compression.

2) 2^{nd} -Order DPCM

2^{nd} -order DPCM decorrelates the signal with the difference between the current and the predicted value.

$$d[n] = x[n] - \hat{x}[n] \quad (1.2)$$

where $x[n]$ is the current sample, so $\hat{x}[n]$ is the value predicted by previous 2 samples. [4]

$$\hat{x}_{dpcm2}[n] = x[n-1] + (x[n-1] - x[n-2]) \quad (1.3)$$

Thus, the expression of typical 2^{nd} -order DPCM is

$$\hat{x}_{dpcm2}[n] = 2x[n-1] - x[n-2] \quad (1.4)$$

2^{nd} -order DPCM not only utilizes the previous value, but also combines the difference of the previous 2 samples to predict the current value.

3) Higher-Order DPCM

To achieve higher accuracy of prediction, higher-order DPCM cascades multiple DPCM stages. Generally, N^{th} -order DPCM algorithm can be expressed as

$$\hat{x}_{dpcmN}[n] = x[n-1] + (\hat{x}_{dpcmN-1}[n-1] - \hat{x}_{dpcmN-1}[n-2]) \quad (1.5)$$

For example, 3^{rd} -order DPCM can be defined as

$$\hat{x}_{dpcm3}[n] = x[n-1] + (\hat{x}_{dpcm2}[n-1] - \hat{x}_{dpcm2}[n-2]) \quad (1.6)$$

Combined with Equation 1.4 and Equation 1.1, this expression can be iterated and deduced to

$$\hat{x}_{dpcm3}[n] = 3x[n-1] - 3x[n-2] + x[n-3] \quad (1.7)$$

which shows that 3^{rd} -order DPCM predicts the current value with the previous 3 samples, and then the difference from the actual value $d[n]$ as Equation 1.2. [4]

However, for large amount of data, the computation will be complicated and thus impossible to be done manually.

4) LNN-Based Decorrelation

LNN-based temporal decorrelation algorithm calculates the value of prediction with neural network. With t taps, the predicted value can be expressed as

$$\hat{x}_{lnn}[n] = b + \sum_{i=1}^t w_i x[n-i] \quad (1.8)$$

By training the network, the values of weights $w_1..w_t$ and bias b can be automatically optimized to achieve minimum MSE. In this way, LNN-based decorrelation can achieve higher prediction accuracy and thus reduce more temporal redundancy. The principle of the algorithm is explained in detail in Section 2.1.

1.2.2 Entropy Encoding

Encoding is to further compress the decorrelated signal, to obtain a compact binary representation. According to Shannon's source coding theorem, the entropy of the original data determines the limitation of SSR.

Entropy is an important concept in information theory, which describes the uncertainty of data values, namely higher entropy represents more decentralized distribution. For a discrete random variable X , there are n possible values x_i , each of which occurs with possibility $P(x_i)$, the entropy is defined as

$$H(X) = - \sum_{i=1}^n P(x_i) \log_2 P(x_i) \quad (1.9)$$

The encoding algorithms can be divided into non-adaptive, semi-adaptive and adaptive methods. The process and parameters of non-adaptive encoding methods are fixed for all inputs, while the adaptive methods modifies itself during the process. In semi-adaptive algorithms, the statistic characteristics of the original data work as compression parameters.

The performance of the encoding techniques can be evaluated with

$$SSR = 1 - \frac{S_{compressed}}{S_{original}} \quad (1.10)$$

where $S_{original}$ and $S_{compressed}$ are the sizes of the bit streams before and after encoding.

According to the principles mentioned before, if the decorrelation process is not effective enough to remove the self-correlation in the signal, there would be a large amount of redundant information for the subsequent encoding process, leading to lower SSR. Therefore, the efficiency of decorrelation dominates the performance of the compression algorithm.

1.3 Datasets

Neural signal is a description of the activities of neurons, can be captured by single electrode at certain position in brain, to record a 1-dimensional signal in time-domain. The amplitude and the pattern of a neural signal can be processed to analyze the neural behavior in the adjacent area around the electrode. [7]

In this work, the datasets from [8] are used as the signals to be processed, which contain neural signals with different spike types and noise level (NL). Specially, 2 types of signals, divided by spike sorting, are used for processing: Type_Easy with less complex characteristics, which makes the spikes more detectable [8], as well as Type_Difficult on the contrary. Both types of neural signals are mixed with noise, of which the overall magnitudes are respectively 0.05, 0.10 and 0.20 of the signals.

Normally, the spikes possess obviously higher amplitudes, and the potential in non-spiking areas are of relatively stable amplitudes. Higher NL increases the overall stochasticity and makes the spikes less recognizable.

Each neural signal consists of multiple spiking and non-spiking areas. Usually, the spikes carry the most of the information about neuron activities. Figure 1.2 displays a short part of the signal of Type_Easy with NL at 0.05, where the black and red arrows point out the spiking and non-spiking areas separately.

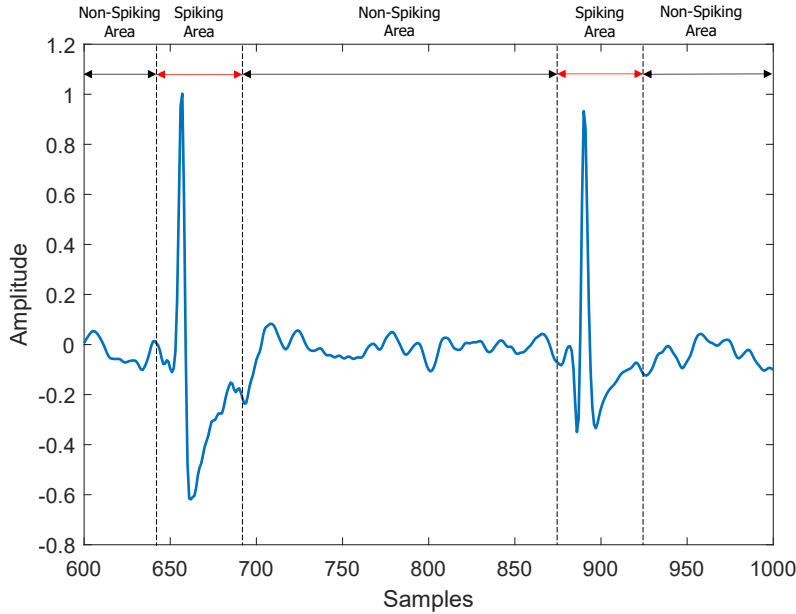


Figure 1.2: Spiking and Non-Spiking Areas

1.4 Thesis Structure

In this thesis, LNN-based decorrelation algorithm is designed, optimized and evaluated for neural signal compression. Compared with DPCM, LNN is a more effective method for decorrelation, achieving higher SSR. After that, the parameters of the trained model is truncated, to reduce the hardware cost. Then a synthesizable hardware implementation of LNN-based decorrelation is designed and verified.

Additionally, datasets that records neural signals with different NL and complexity are used to evaluate the performance of software and hardware implementation of the algorithm.

The thesis includes 3 main chapters.

- Chapter 2: Overview of Compression Algorithm – Detailed explanation of the principles of the compression technique, which consists of LNN-based decorrelation and entropy encoding.
- Chapter 3: Software Implementation – The verification of LNN-based decorrelation in Matlab, possible optimization of hyper-parameters to achieve better performance, and quantization of the model to transplant it from software to hardware.
- Chapter 4: Hardware Implementation – Digital hardware design for the decorrelation algorithm, including hardware design with Verilog HDL, functionality validation by simulation, the analysis of system frequency and power consumption.
- Chapter 5: Conclusion and Related Works – Summary of the results, discussion and related works.

2 Overview of Compression Algorithm

The captured neural signal should be quantified before being processed, namely the conversion of the signal, from analog to digital. As mentioned in Section 1.2, the data compression process includes 2 parts, decorrelation and entropy encoding, which is shown in Figure 2.1.

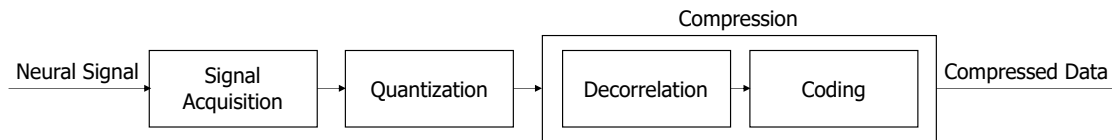


Figure 2.1: Neural Signal Compression

In this chapter, both parts of the compression framework will be explained in detail. As explained in Section 1.2.1, it is a widely used principle for decorrelation algorithms that the value of the latest sample can be predicted by several former samples with a fixed function, and then the difference between the original and predicted values is transmitted to subsequent component to be coded. The coded data can be reformed in a lossless or nearly-lossless way, to obtain the original information.

The SSR of coding algorithm is related to the distribution of data, while the decorrelation process concentrates the distribution. [9] Combined with appropriate decorrelation techniques, the statistical coding algorithms can achieve better performance taking advantage of smaller entropy.

To achieve the minimal power and area consumption in hardware implementation, higher simplicity and efficiency of algorithm are demanded. Individually, the decorrelation and coding techniques are applied for the compression algorithm.

2.1 LNN-Based Decorrelation

Auto- and cross-correlation of the data are considered as temporal and spatial redundancy. For multiple signal channels, cross-correlation plays a significant role,

leading to redundant information across channels. However, in this work, only temporal redundancy is considered and reduced by LNN-based decorrelation.

According to the general algorithm of t^{th} -order DPCM, as Equation 1.5, a polynomial is used to predict the value of the latest sample. In this case, the choice of coefficients is the key to accurate prediction.

LNN-based decorrelation, as expressed in Equation 1.8, provides self-adaptive coefficients, including several trainable weights of neural network (NN) and an additional bias term for adjustment.

Combined with Equation 1.2, the decorrelated data should be

$$d[n] = x[n] - \hat{x}[n] = x[n] - \sum_{i=1}^t w_i x[n-i] - b, \quad n = 1, 2, \dots \quad (2.1)$$

After training the model, the coefficients are optimized. Therefore, LNN-based decorrelation is an evolution of DPCM, reaching higher SSR with limited number of terms in the prediction function.

The workflow of reducing temporal redundancy with LNN-based decorrelation is shown in Figure 2.2. The raw data is firstly digitalized to 8-bit integers, and then processed by a trained LNN model.

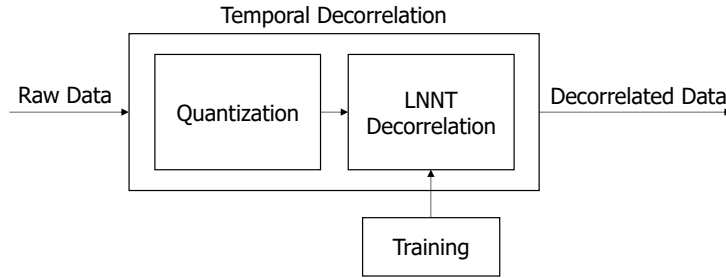


Figure 2.2: LNN-Based Decorrelation

2.1.1 LNN Model

NN is a mathematical model based on neurons, each of which functions as a multiplication between input and weight vectors, combined with an additional adjustment value.

A single-layer LNN is implemented in the prediction process, performing a linear regression. The decorrelation algorithm of a single-channel neural signal can be expressed as

$$\hat{x}[n] = b + WX[n] = b + \sum_{i=1}^t w_i x[n-i] \quad (2.2)$$

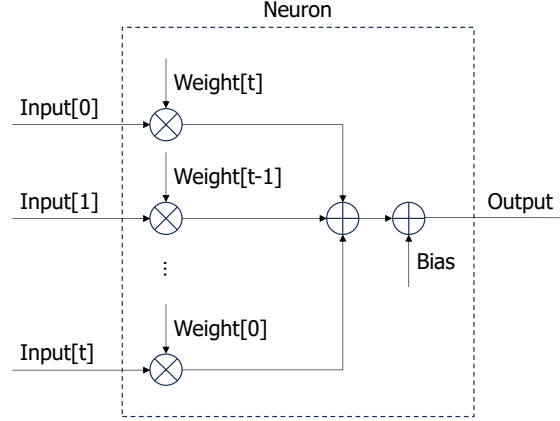


Figure 2.3: Neuron

where W is the weights in the model and X contains t buffered samples of input signal.

Theoretically, larger t leads to more elements in W , which provides more parameters for fine-tuning and therefore higher accuracy in prediction. Nevertheless, excessive number of taps only slightly improves the algorithm capacity, but rises the training time and leads to over-fitting.

However, without non-linearity, stacking LNN layers cannot improve accuracy, because deep LNN is mathematically the same as single-layer LNN. [10] For example, a 2-layer LNN for single channel input signal is represented by Equation 2.3, where the second layer contains only 1 tap, so the weight and bias are both scalar.

$$\hat{x}[n] = b_2 + W_2(b_1 + W_1X[n]) \quad (2.3)$$

whose structure can be simplified as Equation 2.4 so identical to single-layer LNN.

$$\hat{x}[n] = (b_2 + W_2b_1) + W_2W_1X[n] = b_d + W_dX[n] \quad (2.4)$$

The calculation of the mathematical model can be realized by finite impulse response (FIR) filter. The corresponding t -tap FIR filter is shown in Figure 2.4. The input signal is delayed in each step of the filter, to multiply with the corresponding weight. After that, the bias value is added, so that the final result can be produced.

For multiple channels, the prediction process becomes the calculation between matrices. To decorrelate a M -channel neural signal with a t -tap LNN-based decorrelation, a $M \times t$ matrix including t taps of data from each channel is processed with NN, whose weights can be stored in a $t \times M$ matrix W , while the biases are in a $M \times 1$ vector X_t . The predicted value for each channel is accordingly stored in a $M \times 1$ vector.

$$D[n] = X[n] - \hat{X}[n] \quad (2.5)$$

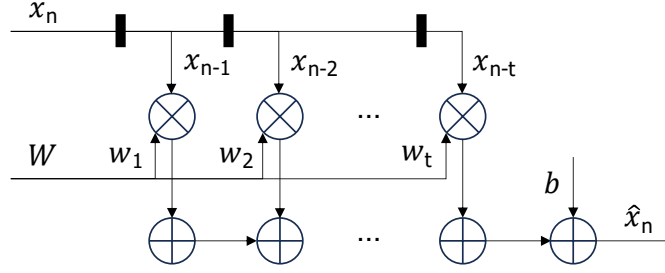


Figure 2.4: FIR Filter

$$\hat{X}[n] = B + WX_t[n - 1] \quad (2.6)$$

In this method, each predicted value is produced by former t data, which means the first t samples cannot be decorrelated. As an optimization, 1st-order and 2nd-order DPCM methods are implemented onto the first t samples.

$$\hat{x}[n] = x[n - 1], \quad n = 2 \quad (2.7)$$

$$\hat{x}[n] = 2x[n - 1] - x[n - 2], \quad n = 3, \dots, t \quad (2.8)$$

2.1.2 Training Process

To determine the optimal weights and bias, the NN model should be trained. During the inference process, the NN executes forward. However, the training process carries on a back-propagation, which updates the values of weights and bias with a learning rate (LR) to minimize the loss function (LF).

In each epoch of training, the NN produces an output signal with the current weights and bias. This result is for loss computation.

In this work, MSE performs as LF of the algorithm, which measures the difference between the predicted and the actual values.

$$L = \frac{1}{N_t} \sum_{n=t+1}^{N_t} \frac{(\hat{x}[n] - x[n])^2}{2} \quad (2.9)$$

where N_t is the total amount of samples used for training. Along with the model getting fitted, the value of LF varies slower and close to 0.

Gradient descent (GD) is the key technique of training, which computes the gradients of LF. For each epoch, the NN parameters are respectively updated using GD method.

$$w_i := w_i - \alpha \frac{\partial L}{\partial w_i} \quad (2.10)$$

$$b := b - \alpha \frac{\partial L}{\partial b} \quad (2.11)$$

Above in Equation 2.10 and Equation 2.11, α is a non-trainable hyper-parameter usually $0 < \alpha < 1$, representing LR. It determines how much the model updates its weights and bias for each epoch. An appropriate choice of LR is important for the training process. The training will be time-consuming if the LR is too small, while large LR may result in divergence of model. Therefore, a varying LR can be implemented for training, which is larger at the beginning and gradually becomes smaller, so that the training can be both efficient and converge.

In this respect, *learnwh* is a Matlab function provided in Neural Network Toolbox. Based on the least mean-square (LMS) learning rule, it adapts the value of LR with the error of the current model. [11] [12] When the error is gradually minimized during training, LR decreases accordingly from the initial α_0 to the optimal value α_{optim} , which ensures the convergence of model.

$$\alpha = \alpha_0 \frac{\partial L}{\partial \alpha} = -\alpha_0 d[n] \quad (2.12)$$

$$\alpha_{optim} = \frac{1}{\sum_{i=1}^t x[i]^2} \quad (2.13)$$

Therefore, the taps at NN input ought to be limited, to avoid the optimal LR being too small. For the same reason, the input signal should not be quantized as Equation 2.14 in the training process.

Epoch is another important hyper-parameter of training. Trained with a suitable number of epochs, the model can predict the values of the data samples precisely. Usually, all of the data are not used in training process. The performance of the model can be estimated with a separate testing set that is different from the training set. Whether the number of epochs is reasonable can be evaluated with variance, namely the difference between the LF of models over testing and training set, and bias. With too few epochs, the model will be under-fitting, featuring low variance and high bias. And excessive training results in over-fitting with high variance and low bias.

2.1.3 Inference Process

In hardware implementation, the input neural signal will be quantified by analog-digital converter (ADC), transformed into series of N -bit samples. For this reason, during the simulation process in software, the raw input data is quantized to N -bit signed integers.

A factor F is involved for this process. The input data is multiplied with the factor, to limit the values in the target range that N -bit integers can describe.

$$x_{norm}[n] = \text{round}(F x_{origin}[n]) \quad (2.14)$$

Since the range of a N -bit signed integer is from -2^{N-1} to $2^{N-1} - 1$, the factors F_{pos} and F_{neg} are defined to set the maximum positive value and the minimum negative value.

$$F_{pos} = \frac{2^{N-1} - 1}{\max(x_{origin})} \quad (2.15)$$

$$F_{neg} = -\frac{2^{N-1}}{\min(x_{origin})} \quad (2.16)$$

Separately, the data can be scaled to avoid exceeding positive and negative limitation.

However, the negative values would exceed the allowed range, if F_{pos} is greater and chosen to multiply with the original data. The same reason makes greater F_{neg} lead to overflow in positive range. Therefore, the factor for scaling should be the minimum between F_{pos} and F_{neg} .

$$F = \min(F_{pos}, F_{neg}) \quad (2.17)$$

The quantization process causes losses, so the training process still uses original data, considering the accuracy of NN model. The quantized data are specially for execution.

The inference of the trained LNN model creates the prediction values of the signal, following Equation 2.2. The realization and optimization of the model will be specified in Chapter 3.

Consequently, the difference between the actual and predicted values, as Equation 2.1, will be transmitted as output.

2.1.4 Decoding

The decoding of LNN-based decorrelation is a recursive process based on NN parameter and the first sample of the decorrelated signal.

It can be deduced from Equation 1.2, the decoded data is the sum of the corresponding predicted and decorrelated data. Only the value of the first sample must be given.

$$x_d[n] = \hat{x}[n] + d[n], \quad n = 1, 2, \dots \quad (2.18)$$

From Equation 1.1, Equation 1.3 and Equation 2.1, the predicted value can be calculated with the given $x[1]$ and the recovered data.

$$\hat{x}[n] = \begin{cases} x[n-1], & \text{if } n = 2 \\ 2x[n-1] - x[n-2], & \text{if } 2 < n \leq t \\ b + \sum_{i=1}^t w_i x_d[n-i], & \text{if } n > t \end{cases} \quad (2.19)$$

2.2 Entropy Encoding

Since the data concentrate around 0 after decorrelation, a concise and effective method of statistical compression is demanded, which represents the original data with symbols of different lengths according to entropy.

For the symbols that appear more frequently, shorter codes are used for representation, while longer ones are for the rare ones. For this reason, a highly-concentrated distribution of samples is important for entropy encoding, so that the optimal average code length can be achieved, approaching to the theoretical limit.

1) Golomb Coding

Golomb Coding (GC) splits each sample into quotient and remainder and encodes them respectively with a unary code and a truncated binary code. [4] For low-power hardware design, GC is an ideal method to lightweight resource consumption. [13]

2) Arithmetic Coding

Arithmetic Coding (AC) encodes the entire message as a real-number fraction, which provides fine-grained control over code lengths, with which the compression can be extremely close to the entropy limit. [4] However, its computational complexity and resource demands are higher. [14]

The SSRs of both GC and AC show similar trends as the concentration of samples to be coded changes. In this work, the compression step is only to show the effectiveness of the decorrelation process. Because its compact structure, GC is implemented as a nearly lossless compression process, as shown in Figure 2.1, The details of GC principles, including both encoding and decoding processes, are interpreted in Appendix A.

3 Software Implementation

In this chapter, the compression techniques, as stated in Chapter 2, will be implemented in software with MatlabR2023b and analyzed according to the results of simulation.

Initially, the LNN-based decorrelation will be proceeded, evaluated with the SSR of GC, and compared with DPCM methods, to validate the effectiveness and efficiency of this algorithm.

After that, the hyper-parameters of the LNN will be fine-tuned, in order to progressively improve its capability.

Additionally, since the compression algorithm is supposed to be launched on hardware, the weights and bias should be truncated, in order to suit for hardware implementation.

The structure of software implementation is displayed in Figure 3.1.

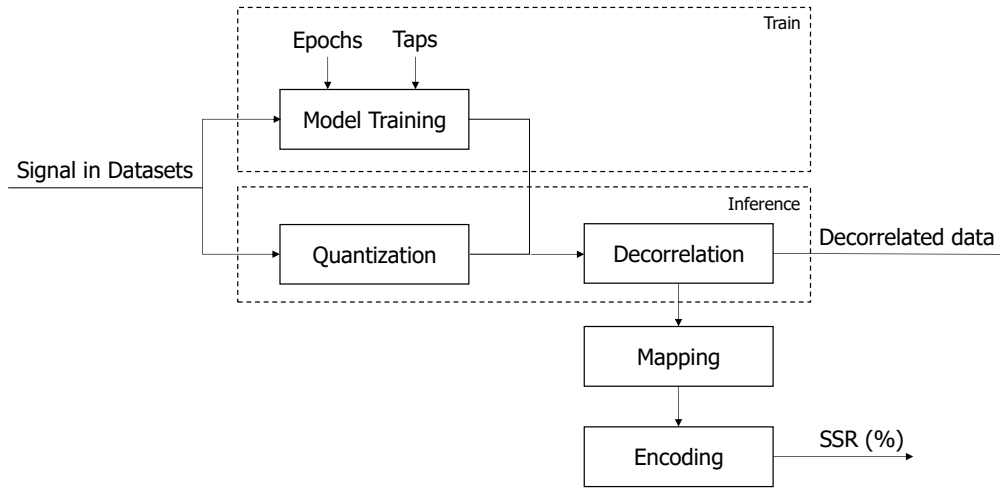


Figure 3.1: Software Design

Practically, non-quantized data are more favorable for training, to obtain higher training speed, which is stated in Section 2.1. Therefore, neural signal from dataset will be transmitted into an LNN model for training, as well as quantized to series of 9-bit signed integers for inference. The inference of the trained LNN model is the decorrelation of the quantized data, to simulate the actual input from ADC module.

After that, the decorrelated data will be encoded with GC method, to produce the criteria for algorithm evaluation, namely the SSR. Then the performance of LNN-based decorrelation can be compared with DPCMs according to their corresponding SSR, to verify its effectiveness.

The optimization of the algorithm contains 2 parts.

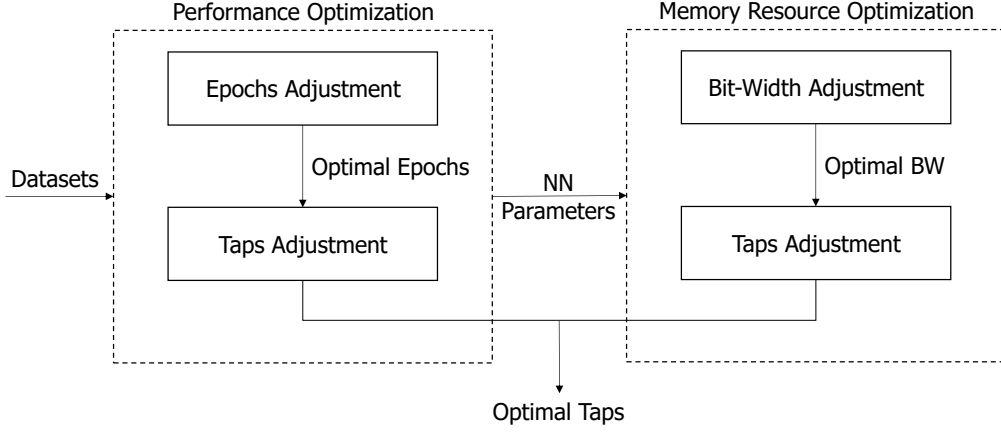


Figure 3.2: Algorithm Optimization

In the aspect of performance improvement, the numbers of the taps in NN and the epochs for training are the 2 most important hyper-parameters of LNN, which directly determines the accuracy of the trained model. Abundant epochs are required to produce a well-fitted model, while too many epochs lead to long training time. On the other hand, more taps provide higher accuracy, but excessive number of taps may result in over-fitting.

As for hardware, the model parameters, namely the values of weights and bias, should be truncated to fixed-point doubles in binary, so that they can be stored in memory, which will naturally affect the performance of the model. To increase the suitability of the decorrelation algorithm on hardware, while maximally preserving the accuracy, the bit-width (BW) of the truncated NN information is supposed to be discussed. Besides, the number of taps will be fine-tuned, to decide the optimal choice, considering the loss caused by transplantation to hardware.

Based on the optimized LNN model, the input signal can be well decorrelated. However, the first t data samples cannot be processed by LNN. For the best performance and real-time capability of decorrelation, 1st-order DPCM will be implemented to decorrelate the second sample using the value of the first one, and 2nd-order DPCM will handle the following $t - 1$ samples.

3.1 Algorithm Realization

3.1.1 LNN-Based Decorrelation Implementation

As it is explained in Section 2.1.1, LNN-based decorrelation is based on a single-layer LNN model, characterized by the weights at the same number as input taps, as well as the bias of model. The software implementation of LNN-based decorrelation includes the training process of the LNN model and the execution of algorithm over the whole input signal.

From each neural signal in the datasets, the first 2000 samples are chosen as training set, to shorten the training time and avoid over-fitting.

The value of learning rate (LR) decides the stride of gradient descending (GD). Using Neural Network Toolbox in Matlab, *learnwh* is assigned as a learning function *learnFcn* in NN. Thus, the LR is self-adaptive, decreases along with the convergence of the model, according to LMS learning rule. In this work, the initial LR α_0 is always set to 0.001, with which the initial speed of training will be sufficiently rapid and prevented from divergence.

According to Equation 2.13, the training set should not be quantized, in order to shorten the training time. Since the quantization step Equation 2.14 maps the raw data approximately in proportion, the LNN-based decorrelation algorithm trained on original data can also correctly decorrelate quantized input, to suit for the hardware design.

To verify of the effectiveness of the algorithm, an LNN model with 32 taps are trained for 20000 epochs, using Type_Easy signal with noise level (NL) at 0.05.

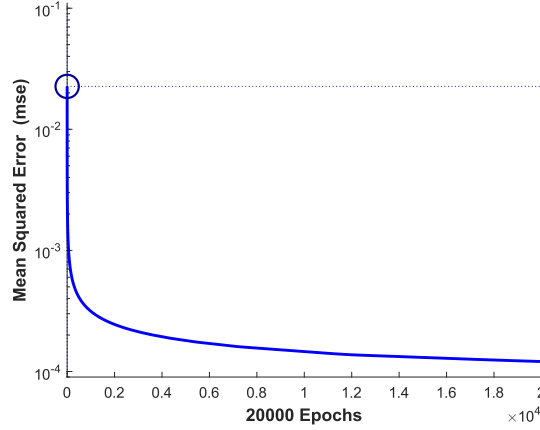


Figure 3.3: Loss (MSE)

The variation of MSE during training is shown in Figure 3.3. The loss function (LF) is reduced rapidly at start, slows down thereafter, and finally reaches

0.000120, which marks the convergence of the model.

The obtained model is then executed to process the quantized data, and the statistical distribution of the resulting signal is as Figure 3.4. According to Section 2.1.3, there are $2^9 = 512$ possible symbols after quantization, distributed in a pseudo-symmetric range.

From the processed dataset, the input signal contains data samples from -243 to 255 , 5.74% of which are at 0. LNN-based decorrelation significantly narrows the range of data, resulting in the range of $-43 \sim 36$, and 70.42% of the samples concentrate at 0, which means that the algorithm realizes a considerable reduction of auto-correlation. Lower entropy and better compression can be therefore achieved.

Over the same dataset, 1st and 2nd-order DPCM concentrates the data into the ranges of $-119 \sim 135$ and $-115 \sim 96$, respectively 26.73% and 38.44% samples assembled at 0. In conclusion, LNN method can effectively decorrelate the input signal, performing remarkably better than DPCM methods.

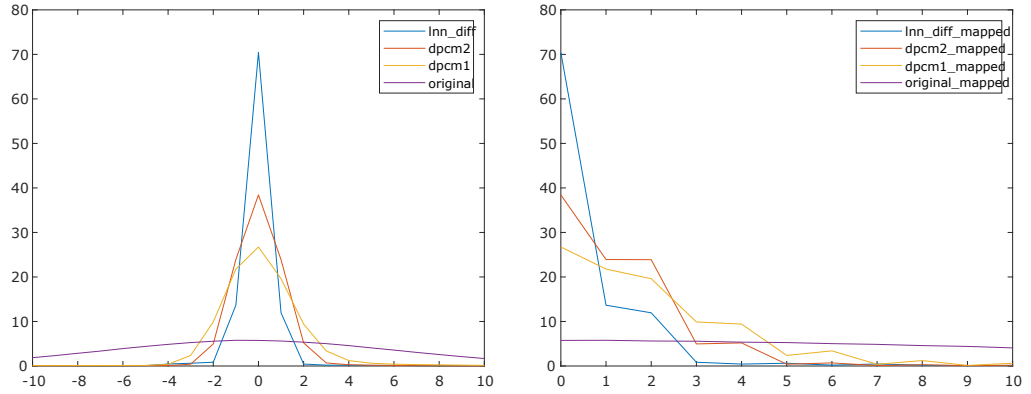


Figure 3.4: Statistical Distribution after LNN-Based Decorrelation

3.1.2 Golomb Encoding Implementation

The performance of decorrelation is evaluated by the SSR that can be reached by coding algorithm. Due to simple implementation, GC is chosen here as an example.

GC method can only process non-negative input due to its division-based construction, as introduced in Appendix A.1. Thus, the decorrelated data should be mapped according to Equation A.1 before being coded. Figure 3.4 shows the distribution of the mapped data.

The decorrelated data are coded separately in groups. Every 50000 samples are divided into a group. The statistical distribution each is counted, in order to choose the adaptive parameter m . In practical software implementation, m can be decided by the percentage of samples at 0. [15] [16]

According to the principles explained in Appendix A, the correspondence between the percentage of zero-centered data and the choice of m can be calculated and listed in Table 3.1. With m , The GC procedure is executed.

Amount of Samples at 0 (%)	m
[0, 5.10)	9
[5.10, 7.40), [10.52, 12.28)	8
[7.40, 8.26), [9.66, 10.52), [12.28, 12.69)	7
[8.26, 9.15), [12.69, 13.05)	6
[9.15, 9.66), [13.05, 13.86)	5
[13.86, 16.51)	4
[16.51, 100]	3

Table 3.1: Choice of Adaptive Parameter m for Golomb Encoding

Consequently, the SSR of the quantized raw data is 40.92%, the 1st and 2nd-order DPCM increases this data to 61.85% and 67.45%, while LNN-base decorrelation results in SSR at 73.97%.

Compared to 1st and 2nd-order DPCM, LNN-base decorrelation relatively lifts the SSR by 19.6% and 9.6%. Therefore, the functionality and the high effectiveness of LNN-base decorrelation are verified.

3.2 Performance Optimization

The optimization of LNN-base decorrelation algorithm, aiming to improve its performance at the software level, includes mainly the tuning of the 2 hyper-parameters, the number of epochs E and taps t .

As proved in Section 3.1.1, the 32-tap LNN model shows good performance after being trained for 20000 epochs. However, most of the weights for higher taps are close to 0, which implies redundancy of parameters, resulting in waste of memory resource.

As observed in experiments, the duration of training increases significantly for excessive epochs, and the performance cannot be visibly improved anymore for

high epochs. So, it is possible to train the model for fewer epochs to make it well-fitted.

In this section, the taps and the epochs are to be optimized, and the signals from more datasets, namely the ones of different type and with different NL, will all be processed as training and testing set, for generalization of algorithm.

3.2.1 Optimal Epochs

To determine the optimal value of E , the effectiveness of the obtained model and the training time should be considered in combination.

In Figure 3.5, the models for different datasets with different noise components are iterated from 1000 to 20000 epochs.

For the signals of different shapes, LNN model behaves differently. Moreover, the performance of LNN-based decorrelation is better for the datasets with lower NL, in which the signal is less blurred by noise components.

Along with the progress of training, the accuracy of the model increases with epochs. However, no improvement can be made after the model being well-fitted, because there are only limited number of parameters in a certain LNN model, which is identical to the number of input taps.

The range of SSR obtained by LNN models with $1 \sim 32$ taps are plotted in the graphs in Figure 3.5. Since SSR doesn't grow proportionally with the number of taps, the average SSR are highlighted in the graphs as reference. It rises decelerately along with the increasing of the number of epochs. Combining the training time and the model accuracy, 4000 epochs can be a reasonable value to take.

Additionally, the SSR of 1^{st} - and 2^{nd} -order DPCM are also plotted for comparison. The minimum of SSR for all epochs and over all datasets, caused by fewer taps, are close to the corresponding SSR of 1^{st} -order DPCM. Referring to Section 1.2.1, a t -tap LNN model is theoretically of similar structure as t^{th} -order DPCM, but with adaptive coefficients and an extra bias. Thus, the performance of t -tap LNN-base decorrelation is expected to be approaching and slightly better than that of t^{th} -order DPCM.

It is interesting that the minimum SSR of Type_Difficult signal with NL at 0.10 is on the contrary. One possible reason is that the training set, namely the first 2000 samples, is not enough representative for the overall characteristics of the signal. It could also be a causation of that the model consists of inadequate taps to well-fit a complex signal, which means even all of the weights and the bias are optimized, the model is still under-fitted. Therefore, it is also important to adjust the number of taps, to obtain an LNN model that is both lightweight and accurate.

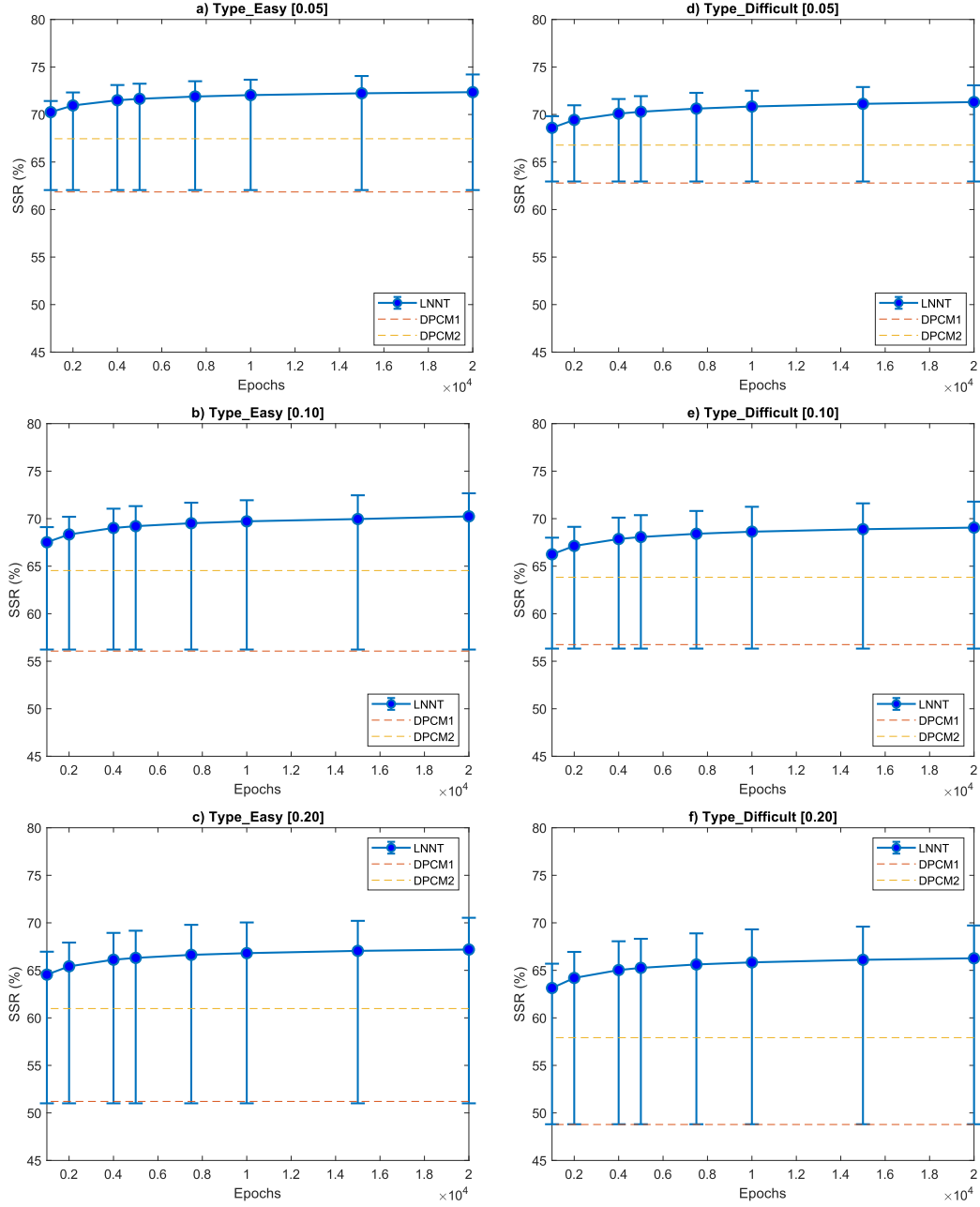


Figure 3.5: Epoch Optimization for Different Signals

3.2.2 Optimal Taps

As mentioned before, a t -tap LNN model gains proximate performance as t^{th} -order DPCM. However, insufficient taps lead to under-fitting, which means that the LNN model cannot inference precisely in this situation, even if it is trained for abundant epochs. On the other hand, excessive taps will result in a waste of storage and computational resource. Meanwhile, the model will obtain memorization over the training set, instead of generalization over the whole signal, if trained for too many epochs.

Trained for 4000 epochs, the performance of the LNN model over each dataset is plotted in Figure 3.6. The SSRs, produced by 2^{nd} -order DPCM decorrelated signal from different datasets, are also displayed as reference.

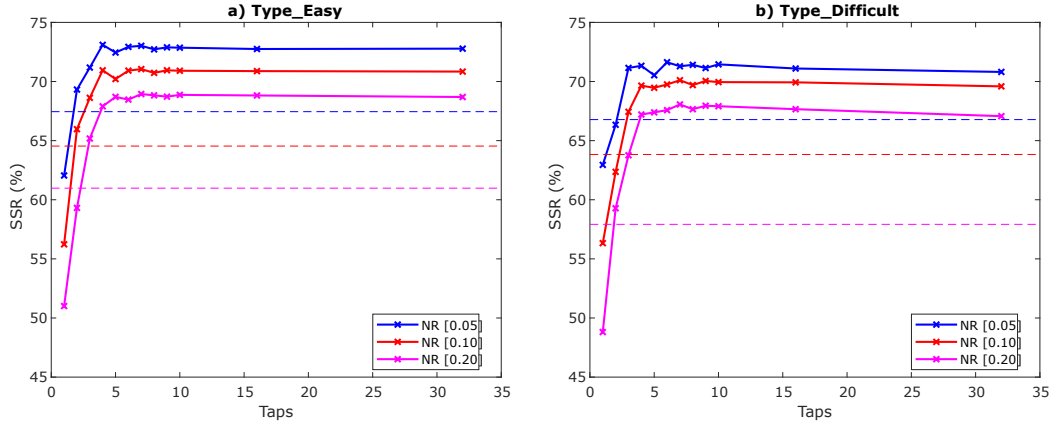


Figure 3.6: Tap Optimization for Different Signals

For fewer-tap LNN-based decorrelation, the SSR of the algorithm increases with the number of taps. At least 3 taps are required in the LNN model, to exceed the performance of 2^{nd} -order DPCM.

However, with sufficient number of taps, more taps will not increase the accuracy of model. Among the operated datasets, SSR fluctuates around the highest value with more than 4 taps, while the maximum appears mostly within $4 \sim 8$ taps. With excessive taps, SSR of the LNN model decreases slightly because of over-fitting.

The phenomenon of over-fitting is more pronounced for more complex signal. As shown in Figure 3.6, the reduction of SSR caused by over-fitting is more obvious in Type_Difficult. Moreover, high NL can also compound the loss of SSR.

Therefore, the optimal choice of t should be within $4 \sim 8$. This hyper-parameter will be further tuned in section 3.4.

3.3 Memory Recourse Optimization

In the progress of algorithm optimization, the difference between the computation on software and hardware platforms should be taken into consideration.

In Matlab, the numeric variables are stored as 64-bit doubles, which is unsatisfactory in hardware. In practical, a loss in accuracy will be caused by the characteristics of logic circuit, where the data is stored as fixed-point binary with fewer bits.

To improve the hardware suitability of the LNN-based decorrelation method, the algorithm is executed simulating the hardware situation. Specifically, the BW of the weights and bias in memory is expected to be discussed.

Trained for 4000 epochs, the obtained NN parameters are saved. Apart from the original version, the parameters are transformed into 4, 8, 12, 16 and 32-bit fixed-point signed doubles in binary, constructed according to Table 3.2.

Total BW (bit)	Sign (bit)	Integer (bit)	Fraction (bit)
4	1	2	1
8	1	2	5
12	1	2	9
16	1	4	11
32	1	8	23

Table 3.2: Construction of Truncated Parameters

Then the parameters with different BWs are loaded into the model, with which the algorithm will be run again. In this way, the optimal BW can be decided according to the loss of SSR, balancing the memory consumption and model accuracy.

Figure 3.7 shows the variation of SSR along with taps, which are separately produced by the LNN models that are original and truncated to 4, 8, 12, 16, 32 bits. In principle, the weights and bias with wider BW improves the accuracy of model, but also raises memory consumption. It can be seen in the graphs that the 4-bit model performs visibly worse than the original one, and the 8-bit one derives slightly, especially for the datasets with higher NL. Nevertheless, the SSR of the data compressed by 12, 16 and 32-bit NN parameters are very closed to the original results. Considering the memory space, it will be optimal to truncate the weights and bias into 12-bit fixed-point doubles.

Specially, the SSR of 8-bit LNN model will deviate more significantly from the original, because the higher-tap models gain higher precision, but more errors will

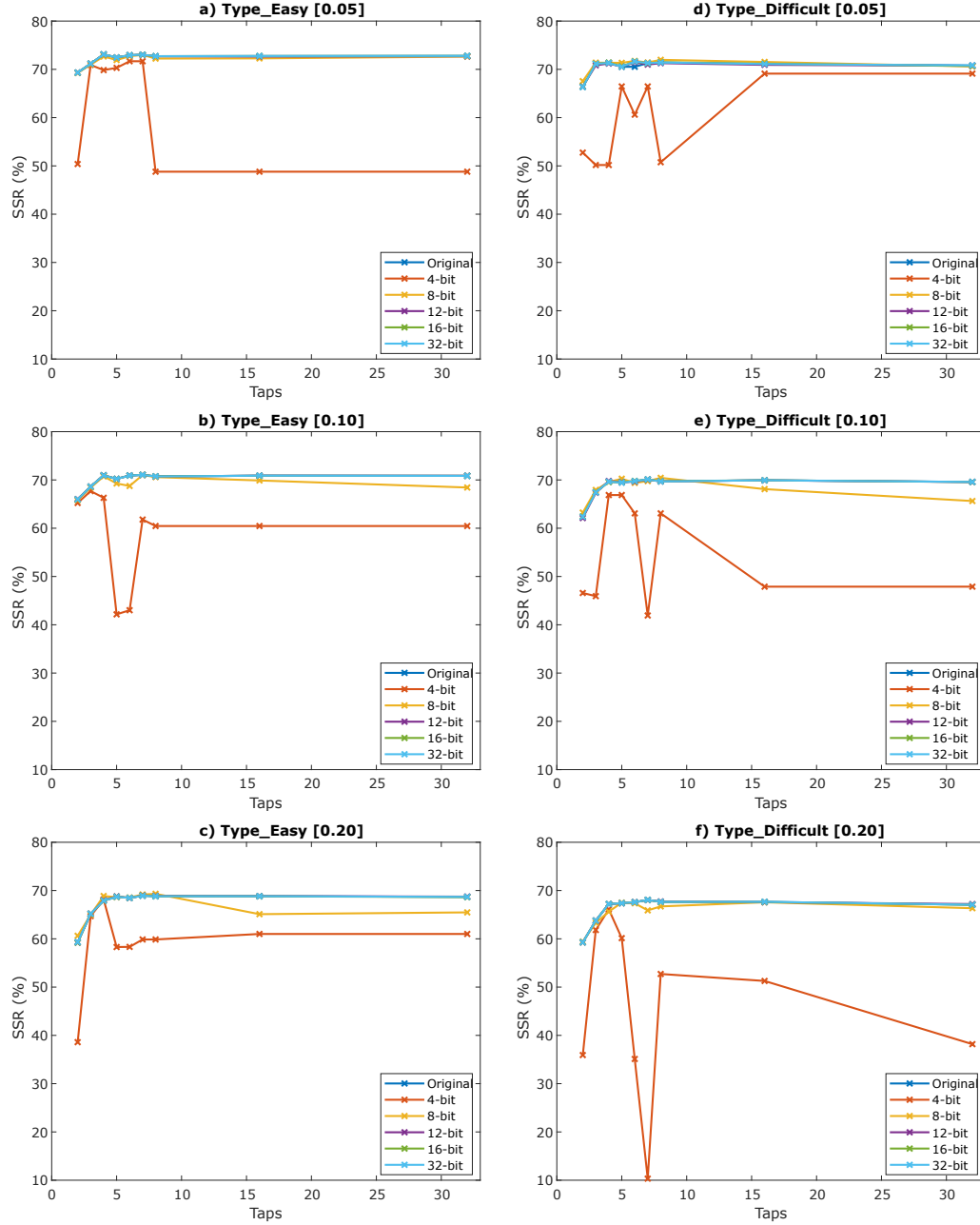


Figure 3.7: BW Optimization for Different Signals

be accumulated when more deviated parameters are used for calculation. For this reason, applying too many taps in model is inappropriate.

According to the conclusion drawn in Section 3.2.2, the optimal number of taps in the LNN model is $4 \sim 8$. In this section, the models with 12-bit truncated parameters can still closely realize the original performance. Thus, the hyper-parameter t should be decided as 4, considering both the deviation and memory space.

Fixed-point arithmetic avoids the accumulation of floating-point errors, so that the calculation can be more stable. It is interesting that the model with more taps can sometimes perform better with lower-bit parameter, while the SSR produced by original model decreases for over-fitting. A possible reason is that 8-bit quantization reduces the range of the parameters. It is similar to the drop-out effect, by which the model is regularized and denoised. A part of the tiny weights can be rounded-off, so that the weak-related characteristics are ignored, which improves the decorrelation performance by reducing the reliance on unnecessary information.

Eventually, the hardware-suited LNN model should consist of 12-bit parameters for 4 taps, trained for 4000 epochs.

3.4 Real-Time Processing

Selected the 4-tap LNN model, the neural signal can be decorrelated efficiently. In accordance with the previous discussions, each decorrelated value should be drawn from the former t data samples. It means that the designed LNN-based decorrelation method cannot process the first 4 data samples.

In the previous discussions about optimization, the first t was actually not decorrelated, but copied directly from the input. Moreover, there will be t -sample's latency, if the first t samples in output are abandoned as invalid. Thus, apart from LNN-based decorrelation, the 1^{st} and 2^{nd} -order DPCM methods are also merged into a hybrid decorrelation algorithm, so that it can achieve both high precision and real-time processing.

Consequently, the predicted data will be

$$\hat{x}[n] = \begin{cases} 0, & \text{if } n = 1 \\ x[n-1], & \text{if } n = 2 \\ 2x[n-1] - x[n-2], & \text{if } 2 < n \leq t \\ b + \sum_{i=1}^t w_i x_d[n-i], & \text{if } n > t \end{cases} \quad (3.1)$$

The simulated SSR of the neural signals compressed with this hybrid strategy and GC are displayed in Table 3.3.

Dataset Type	NL	SSR by GC (%)
Easy	0.05	73.08
Easy	0.10	70.97
Easy	0.20	67.97
Difficult	0.05	71.20
Difficult	0.10	69.77
Difficult	0.20	67.19

Table 3.3: Performance of Optimized Algorithm

4 Hardware Implementation

A LNN-base decorrelator, as designed and optimized in Chapter 3, will be parallelized to process multi-channel signal and implemented in hardware in this chapter.

The hardware implementation of the LNN-base decorrelator mainly includes a 9-bit input from ADC, an inner static random-access memory (SRAM) providing 12-bit parameters, the processing blocks that executes the hybrid decorrelation algorithm as explained in Section 3.4, and a 9-bit binary output. Additionally, there are also the system clock and reset signals, the activation signal for each channel, the validation signal of the decorrelator, as well as the index signal that points out the currently processed channel.

The functionality of the designed component will be verified by simulation, for which the reference output will be generated by programs in Matlab. After that, the LNN-base decorrelator will be synthesized and the frequency of SCLK will be optimized, in order to ensure the successful execution of decorrelation and balance the performance and the power consumption.

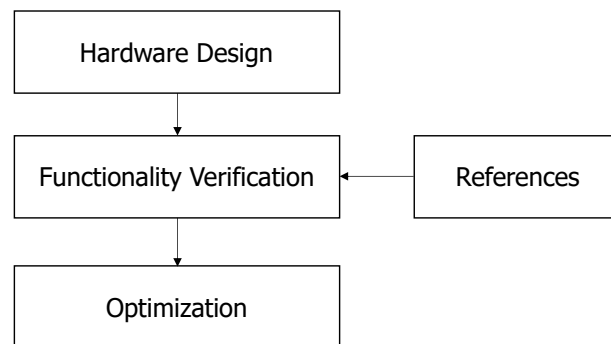


Figure 4.1: Hardware Implementation

4.1 Hardware Design

The designed hardware module is composed by multiple LNN-base decorrelation components. The weights and bias of the LNN are stored in SRAM, which communicates with the top module. Initially, the LNN parameters are loaded as

input. After receiving the activation signal, each sample of input data, including a 12-bit fixed-point double and the index of channel, will be distributed to the corresponding LNN-base decorrelator to be processed separately.

4.1.1 Multi-Channel Processing

Figure 4.2 shows the structure of the hardware design.

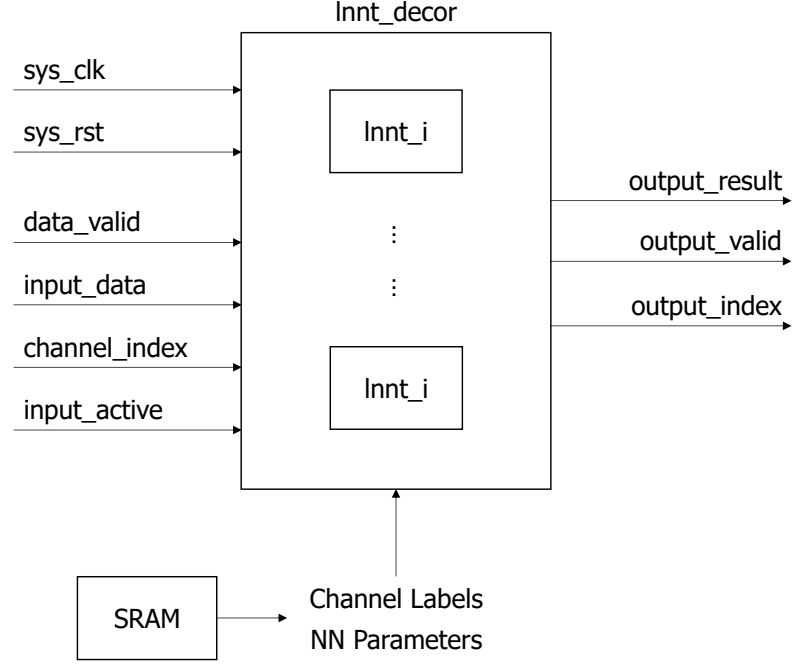


Figure 4.2: Top Module of the LNN-Based Decorrelator

There are 3 parameters to configure the system.

- `NUM_CH = 16`: The number of signal channels.
- `NUM_TAPS = 4`: The taps in the LNN model.
- `NUM_WIDTH = 12`: The BW of parameters in the LNN model.

The input ports of the top module `lnnt_decor` are listed as below.

- `sys_clk`: Global clock signal.
- `sys_rst`: Global reset signal that initializes the system when being set to 0.
- `data_valid`: The signal that enables the input data to be processed when it is pulled-up,

- `input_data`: The neural signal captured by ADC, represented by 9-bit signed integer.
- `index`: 6-bit integer that points out the number of channel, to which the current input data should be distributed.
- `input_active`: The bus of `NUM_CH` activation signals corresponding to each channel. When `input_active[x]` is set to 1, the channel `x` will be activated.

The files stored in SRAM are read as input signals when the system is initialized. After that, the loaded LNN parameters will be saved in inner register arrays.

- `weights_ch<x>_taps<y>`: The values of weights in LNN model for the taps $0 \sim \text{NUM_TAPS} - 1$ in the channels $0 \sim \text{NUM_CH} - 1$.
- `bias_ch<x>`: The bias of LNN model for each channel.
- `list_ch<x>`: The index code for each channel.

The output ports export the processed data and relative information.

- `output_result`: The decorrelated data in form of 9-bit binary integer.
- `output_valid`: The output validation signal.
- `output_index`: The index of channel for output data.

When the `data_valid` signal is set to 1, the system starts to operate. For each synchronized clock cycle, the input information contains 2 parts, the input data and its index. According to the index codes read from the configuration file, the input data can be distributed to the matched channel.

If the activation signal, namely the corresponding bit in `input_active` bus is at high-level, the input data will be buffered and processed in the decorrelation sub-module.

Thereafter, the decorrelated data and the validation signal will be produced. Any one of the multiple channels being executed makes the overall output signal valid.

4.1.2 Decorrelation Unit

Each unit of the LNN-base decorrelator realizes the calculation of NN for an individual channel.

As verified and optimized in Chapter 3, 4 taps of data should be buffered inside, to be processed by LNN model. Before enough number of samples accumulated,

the data can be processed by DPCM. Thus, the principle of the block can be expressed by Figure 4.3.

An integer cnt is used to record the number of input samples in the current unit. The first sample is abandoned, while the second one is processed by 1st-order DPCM, and the following 2 samples are decorrelated with 2nd-order DPCM. Received enough taps of data, the predicted data can be calculated with LNN-base decorrelation.

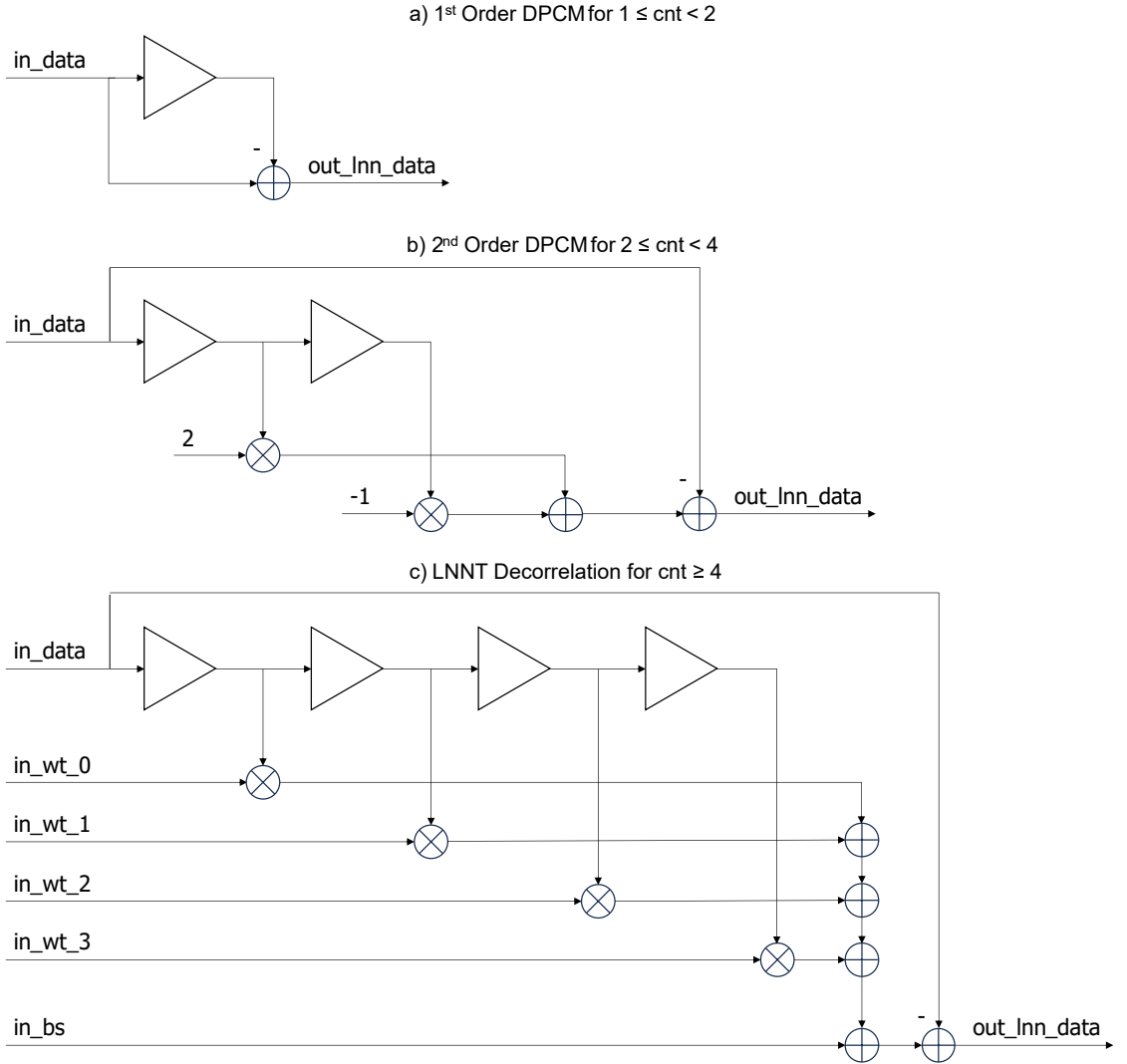


Figure 4.3: Hybrid Decorrelation Unit

4.2 Functionality Verification

The functionality of the designed module can be verified by simulation, using Cadence NCSIM tool. Before simulation, the HDL-check is run for the designed components, which makes sure that the design is valid to be functioned.

For multiple processing channels, the neural signal from different datasets are used for simulation. After training the 4-tap LNN model for 4000 epochs, the parameters of the network are quantized to 12-bit signed fixed-point doubles and written to file.

Since there are multiple channels, the simulated input series are the samples from different datasets in interleave. Accordingly, the output for reference can be generated. Using the quantized input neural signal and exported LNN parameters, the theoretical output data can be calculated.

A testbench for the component is designed to make comparisons between the result of software and hardware decorrelator, as shown in Figure 4.4.

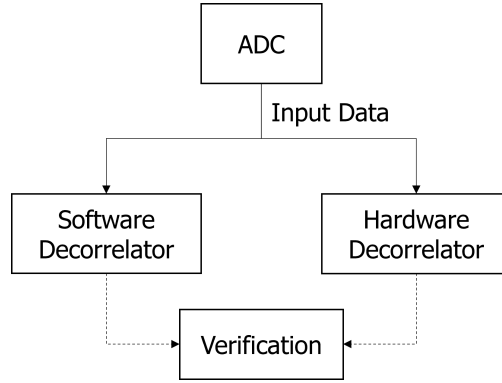


Figure 4.4: Verification Scheme

For the cases that the channels are fully and partly activated, the testbench provides the system clock, reset and other input signals from files as excitation sources. The output data of the module are compared with the references, and error information will be provided when the results are different.

Consequently, it is proved that the hardware and software decorrelators possess the identical functionality, and the results can be generated correctly.

4.3 Performance Analysis

The LNN-base decorrelator is synthesized with Synopsys Design Compiler in this section, and the speed and power consumption of the hardware component is estimated under 28nm node.

The frequency of system clock can be adjusted in the constraint files. Since the real-time neural signal is transferred through ADC with its system clock of 200kHz, and the decorrelation can be executed every clock cycle, the lowest frequency of the system should be 200kHz.

Scanning from 200kHz, the total cell area and power consumption of the system are displayed in Figure 4.5.

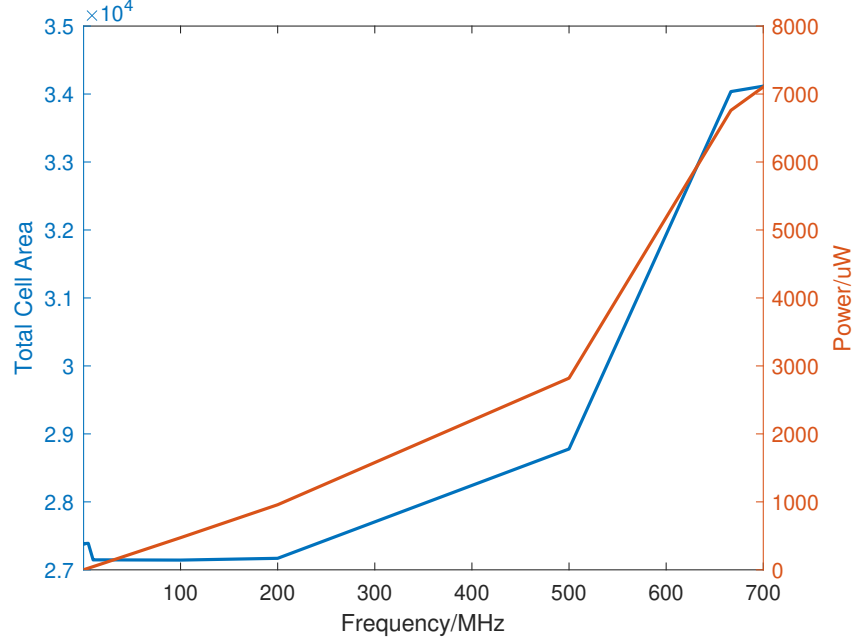


Figure 4.5: Area and Power Consumption

For low frequency, the total cell area varies slightly, while it rises for more than 200MHz. After 600MHz, the area reaches the limitation and grows more slowly.

The global operating voltage is 0.9V. The power consumption of the system consists of 2 parts, the cell internal power and net switching power. For the system clock of less than 500MHz, the power consumption is approximately proportional to the frequency, including 91% cell internal power and 9% net switching power. After that, the power increases faster and the percentage of net switching power rises to 25%, until it reaches the limitation.

5 Conclusion and Related Works

5.1 Summary

This thesis proposes an LNN-based neural signal decorrelation method, implemented by software and hardware.

Initially, the principles of the core techniques are introduced. The theory of LNN-based decorrelation is explained, including the basic construction, the training and inference processes. Besides, entropy encoding is used as an expression of the effectiveness of decorrelation.

After that, the proposed compression techniques are implemented on Matlab platform. The LNN-based software decorrelator is realized and verified, showing significant improvement in signal decorrelation. LNN-based decorrelation narrows the data range and achieves SSR of 73.97%, increased by 19.6% and 9.6% compared to 1st- and 2nd-order DPCM methods.

Next, the key hyper-parameters are optimized to balance the model performance and training time. The first step is to tune the numbers of training epochs and taps in model, aiming to good generalization and avoid over-fitting. The second step is to simulate the practical situation in hardware, the weights and bias parameters of the model are truncated, transforming to fixed-point doubles, and the number of taps is further fine-tuned, to ensure efficient memory utilization with minimal accuracy loss. The experiments suggest that the 4-tap LNN model trained for 4000 epochs, consisting 12-bit fixed-point parameters, will provide the the best performance and memory consumption.

A hardware implementation of the LNN-based decorrelator is presented. Using on-chip SRAM to store the quantized weights and biases, the design parallelizes multiple decorrelation units, to handle multi-channel signals. Based on channel indexing and activation signals, the input signals are distributed to the units for processing. For real-time functionality, the samples at the beginning are decorrelated with DPCM methods. The design is verified through simulation, confirming that the software and hardware results match precisely. Finally, the synthesis under 28nm node analyzes area and power consumption of the design, which vary with the system frequency, to demonstrate the feasibility for embedded deployment.

5.2 Discussion

In this work, a lightweight neural-network-based decorrelator is implemented for neural signal compression.

The training process of LNN model uses only the first 2000 samples in each dataset, which ensures quick convergence and low complexity. However, it also leads to that the prediction is mainly for noise, because 2000 samples cannot contain a complete spike.

LNN parameters are quantized as 12-bit fixed-point numbers to improve memory usage, which is not a natural word length in hardware and may lead to slight overhead in arithmetic operations. Nevertheless, it balances the prediction accuracy and resource consumption.

In addition, the number of activated channels in hardware can be configured with the activation signal. Considering the adaptability of LNN model to different signal conditions and performance requirements, it would be better to have the number of taps configurable as well.

Overall, the design in this thesis achieves efficient decorrelation and allows practical application.

5.3 Related Works

1) Traditional Neural Signal Decorrelation

Decorrelation methods rely mainly on signal prediction, producing the difference between the predicted and original values for coding. For example, delta encoding stores the differences between adjacent samples in neural signal to reduce data variance, typically combined with subsequent Huffman coding. [17] Moreover, DPCM predicts each sample using previous values, which reduces short-term redundancy but struggles with non-linear noise. [4]

2) Adaptive Coding and Entropy Encoders

Adaptive arithmetic coding dynamically updates its probability model during encoding. Context-based adaptive arithmetic coders, such as CABAC [18], realizes non-stationary data distribution without preliminary pass. In neural-network compression, specialized adaptive coders like DeepCABAC [19] apply context-adaptive arithmetic coding to quantized weights, combining quantization with context models, to compress deep networks up to $63\times$ while preserving accuracy.

3) Neural Networks in Hardware

Deep compression introduces a three-stage pipeline, namely pruning, trained quantization and lossless Huffman coding. Therefore, large NN models can be realized by on-chip modules, which significantly cuts inference energy and memory requirements. [20] Besides, an FPGA-based hybrid architecture is designed in [21], combining compressive sensing with a multi-layer perceptron predictor, to compress electrocorticography (ECoG) signals. 3 configurations—full FPGA MLP are explored, ARM+FPGA heterogeneous deployment, and sparse compression—finding that the ARM+FPGA hybrid achieves efficient low-latency, low-power compression and reconstruction on edge devices, which highlights how modest neural predictors can be effectively offloaded to reconfigurable logic in multi-core edge systems.

Appendix A

Golomb Coding

A.1 Encoding

The decorrelated data, namely the difference between the acquired data and the series predicted by LNN, will be coded with GC method, whose workflow is illustrated in Figure A.1.

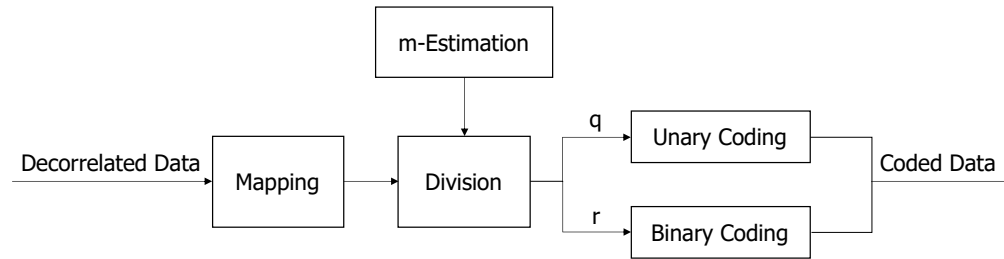


Figure A.1: Golomb Encoding

The data to be coded are expected to be mapped into non-negative range in interweave, as Equation A.1.

$$d_m[k] = \begin{cases} 2d[k], & \text{if } d[k] \geq 0 \\ -2d[k] - 1, & \text{if } d[k] < 0 \end{cases}, \quad k = 1, 2, \dots \quad (\text{A.1})$$

To compress a non-negative number n with GC method, a adaptive parameter m should be decided, with which 3 necessary quantities of GC are calculated.

m divides the input data n into a quotient q and a remainder r , which will be coded respectively in unary and binary in subsequent process.

$$q = \lfloor \frac{n}{m} \rfloor \quad (\text{A.2})$$

$$r = n - qm \quad (\text{A.3})$$

q is coded as the first part of GC, namely the higher $q + 1$ bits. There are supposed to be series of 1's, of which the total number is equal to q and can be 0. This section is always ended with a 0.

For the other part, since r is within the range from 0 to $m - 1$, the bit-width (BW) that is sufficient for encoding r should be

$$c = \lceil \log_2 m \rceil \quad (\text{A.4})$$

This part contains the binary type of r of certain BW. When m is not a power of 2, the values of r from 0 to $2^c - m - 1$ can be encoded in $c - 1$ bits, while the rest are coded in c bit each. Otherwise, all of the possible values of r should be coded in c bits.

For example, if $n = 43$ and $m = 14$, it can be easily known that $c = 4$, and the data can be encoded as

$$\begin{aligned} q = 3 &\Rightarrow 1110 \\ r = 1 &\Rightarrow 001 \\ n = 43 &\Rightarrow 1110|001 \end{aligned} \quad (\text{A.5})$$

then the input data is compressed into a $(q + c)$ or $(q + c + 1)$ -bit binary code.

It can be told that the choice of m is critical. Practically, larger m increases the required bit length to encode n , while smaller m leads to longer encoded sequences. An optimal m maximizes the SSR of the coding process.

Run-length encoding (RLE) is the key method to determine the value of parameter m . RLE represents a binary data sample as several (L, a) pairs, where L is the number of consecutive identical value $a = 0$ or 1 within the binary string.

For GC, only K pairs with $a = 0$ are taken into consideration. The mean RL can be calculated as

$$\bar{L} = \frac{1}{K} \sum_{j=1}^K L_j \quad (\text{A.6})$$

Consequently, the optimal adaptive parameter of GC can be estimated.

$$m = 2^{\lceil \log_2(1+\bar{L}) \rceil} \quad (\text{A.7})$$

A.2 Decoding

The encoding process of GC produces a successive bit-stream, which makes the detection of elements a significant task of decoding.

As an LLC algorithm, the decoding process of GC includes 2 parts, recovering the original data losslessly.

As mentioned formerly, the quotient q is encoded in unary. So, from the first bit of the code, the number of 1's should be count, until the first 0 appears. The length of this part is

$$L_q = A + 1 \quad (\text{A.8})$$

This number A is exactly the value of the original q .

The rest of the code is encoded from r . If m is a power of 2, the length of the second part should be fixed to c , so the c bits after the 0 at the end of the first part represents the value of the original r .

$$L_r = c \quad (\text{A.9})$$

However, if m is not a power of 2, a trial value R should be taken, denoting the $c - 1$ bits after the 0 at the end of the first part. The actual length of the coded r can be identified by

$$L_r = \begin{cases} c - 1, & \text{if } R \leq 2^c - m - 1 \\ c, & \text{if } R > 2^c - m - 1 \end{cases} \quad (\text{A.10})$$

After correcting the length of coded r , true value of r can be finally recovered.

Bibliography

- [1] Karim G. Oweiss. *Brain-Machine Interface: Overview*, pages 12–19. Springer New York, New York, NY, 2022.
- [2] Claude Bedard, Charlotte Piette, Laurent Venance, and Alain Destexhe. Extracellular and intracellular components of the impedance of neural tissue. *Biophysical Journal*, 121(6):869–885, 2022.
- [3] Claude Bédard, Serafim Rodrigues, Noah Roy, Diego Contreras, and Alain Destexhe. Evidence for frequency-dependent extracellular impedance from the transfer function between extracellular and intracellular potentials. *Journal of Computational Neuroscience*, 29(3):389–403, 2010.
- [4] D. Salomon, D. Bryant, and G. Motta. *Handbook of Data Compression*. Springer London, 2009.
- [5] K. Sayood. *Lossless Compression Handbook*. Communications, Networking and Multimedia. Academic Press, 2002.
- [6] Qier Ma, Liyuan Guo, Seyed Mohammad Ali Zeinolabedin, and Christian Mayr. Ultra-low power and area-efficient hardware accelerator for adaptive neural signal compression. In *2021 IEEE Biomedical Circuits and Systems Conference (BioCAS)*, pages 1–4, 2021.
- [7] Ramana Vinjamuri. *Advances in Neural Signal Processing*. IntechOpen, Rijeka, Sep 2020.
- [8] R. Quian Quiroga, Z. Nadasdy, and Y. Ben-Shaul. Unsupervised spike detection and sorting with wavelets and superparamagnetic clustering. *Neural Computation*, 16(8):1661–1687, 2004.
- [9] Xaq Pitkow and Markus Meister. Decorrelation and efficient coding by retinal ganglion cells. *Nature Neuroscience*, 15(4):628–635, 2012.
- [10] Pierre Ablin. Deep orthogonal linear networks are shallow, 2020.
- [11] Bernard Widrow and Samuel D. Stearns. Adaptive signal processing. In *Adaptive Signal Processing*, 1985.

- [12] Bernard Widrow and Marcian E. Hoff. (1960) bernard widrow and marcian e. hoff, "adaptive switching circuits," 1960 ire wescon convention record, new york: Ire, pp. 96-104. In *Neurocomputing, Volume 1: Foundations of Research*. The MIT Press, 04 1988.
- [13] Priyanka Kalode and Richa Khandelwal. Test data compression based on golomb coding and two-value golomb coding. *Signal Image Processing : An International Journal*, 3, 04 2012.
- [14] Jyotika Doshi and Savita Gandhi. Improved performance of arithmetic coding by extracting multiple bits at a time. *International Journal of Engineering Research & Technology (IJERT)*, 1(8):–, October 2012.
- [15] Liyuan Guo, Annika Weiße, Seyed Mohammad Ali Zeinolabedin, Franz Marcus Schüffny, Marco Stolba, Qier Ma, Zhuo Wang, Stefan Scholze, Andreas Dixius, Marc Berthel, Johannes Partzsch, Dennis Walter, Georg Ellguth, Sebastian Höppner, Richard George, and Christian Mayr. 68-channel neural signal processing system-on-chip with integrated feature extraction, compression, and hardware accelerators for neuroprosthetics in 22 nm fdsoi. *Frontiers in Neuroscience*, 18, 2024.
- [16] Liyuan Guo, Seyed Mohammad Ali Zeinolabedin, Franz Marcus Schüffny, Annika Weiße, Stefan Scholze, Richard George, Johannes Partzsch, and Christian Mayr. A 16-channel real-time adaptive neural signal compression engine in 22nm fdsoi. In *2023 21st IEEE Interregional NEWCAS Conference (NEWCAS)*, pages 1–5, 2023.
- [17] Daniel Wendler, Jan Körber, Daniel De Dorigo, Roman Willaredt, Matthias Kuhl, and Yiannos Manoli. Data rate reduction in intracranial digital neural probes using delta encoding. In *2024 IEEE Biomedical Circuits and Systems Conference (BioCAS)*, pages 1–5, 2024.
- [18] Rui Song, Dong Liu, Houqiang Li, and Feng Wu. Neural network-based arithmetic coding of intra prediction modes in HEVC. *CoRR*, abs/1709.05737, 2017.
- [19] Simon Wiedemann, Heiner Kirchhoffer, Stefan Matlage, Paul Haase, Arturo Marbán, Talmaj Marinc, David Neumann, Tung Nguyen, Ahmed Osman, Detlev Marpe, Heiko Schwarz, Thomas Wiegand, and Wojciech Samek. Deepcabac: A universal compression algorithm for deep neural networks. *CoRR*, abs/1907.11900, 2019.

- [20] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015.
- [21] Ritu Ranjan Shrivastwa, Vikramkumar Pudi, and Anupam Chattopadhyay. An fpga-based brain computer interfacing using compressive sensing and machine learning. pages 726–731, 07 2018.

